

From Redis to Apache KVRocks (it's not about the license)

Redis

- In-memory key–value database, open source
- Used for: caching, sessions, queues, events, streams, etc.

Good: all data is in memory

Bad: all data needs to fit in memory

What is Apache KVRocks?

- A key value NoSQL database
- Uses **RocksDB** as storage engine and
- Is **compatible with Redis** protocol
- Open source, Apache license, 4k stars

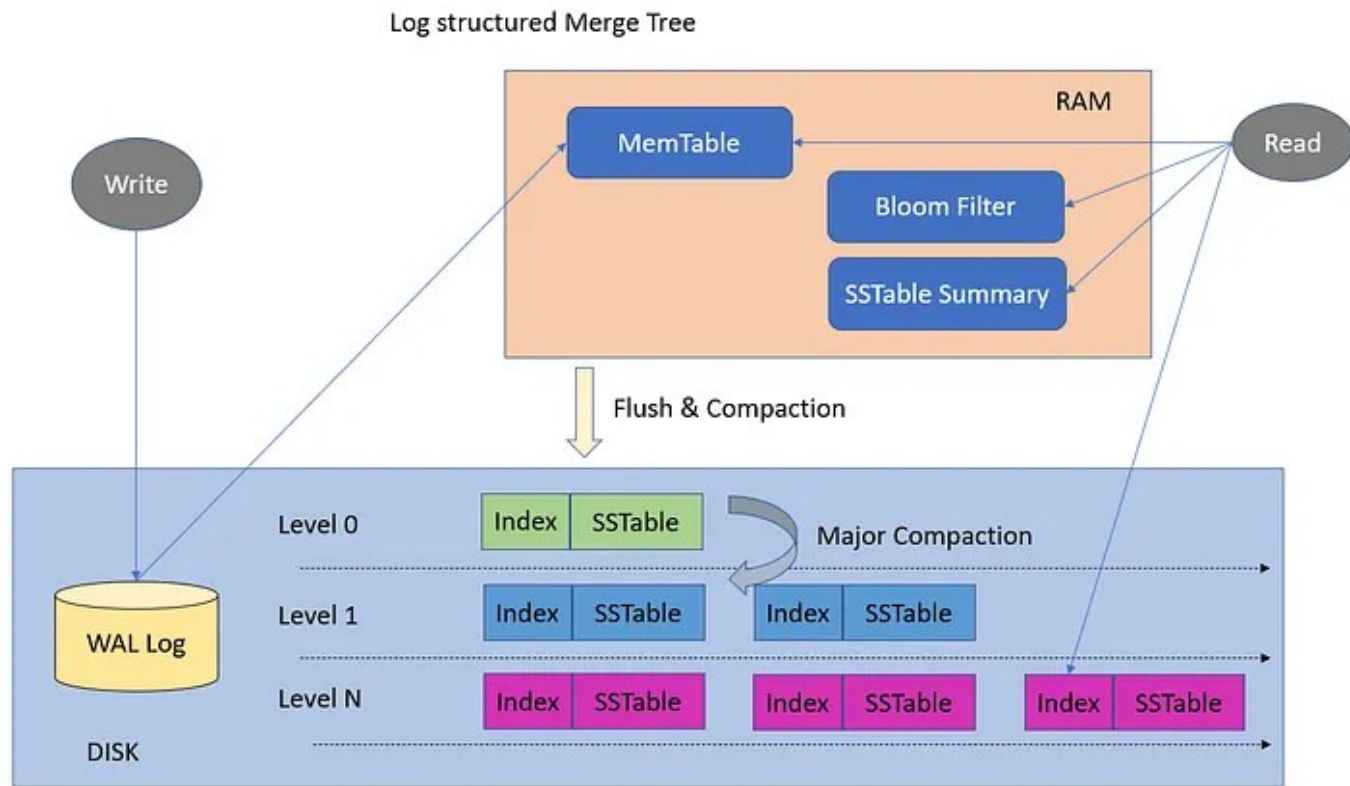
Basically Redis on disk and ram

Can store data larger than ram, does not require eviction

RocksDB

- High performance embedded database for key-value data
- Keeps hot data in **memory** (block cache)
- Stores all data compressed on **disk** using LSM tree data structure
- Provides transactions, snapshots, expiry with TTL
- Developed by Meta in C++, open source, 30k stars

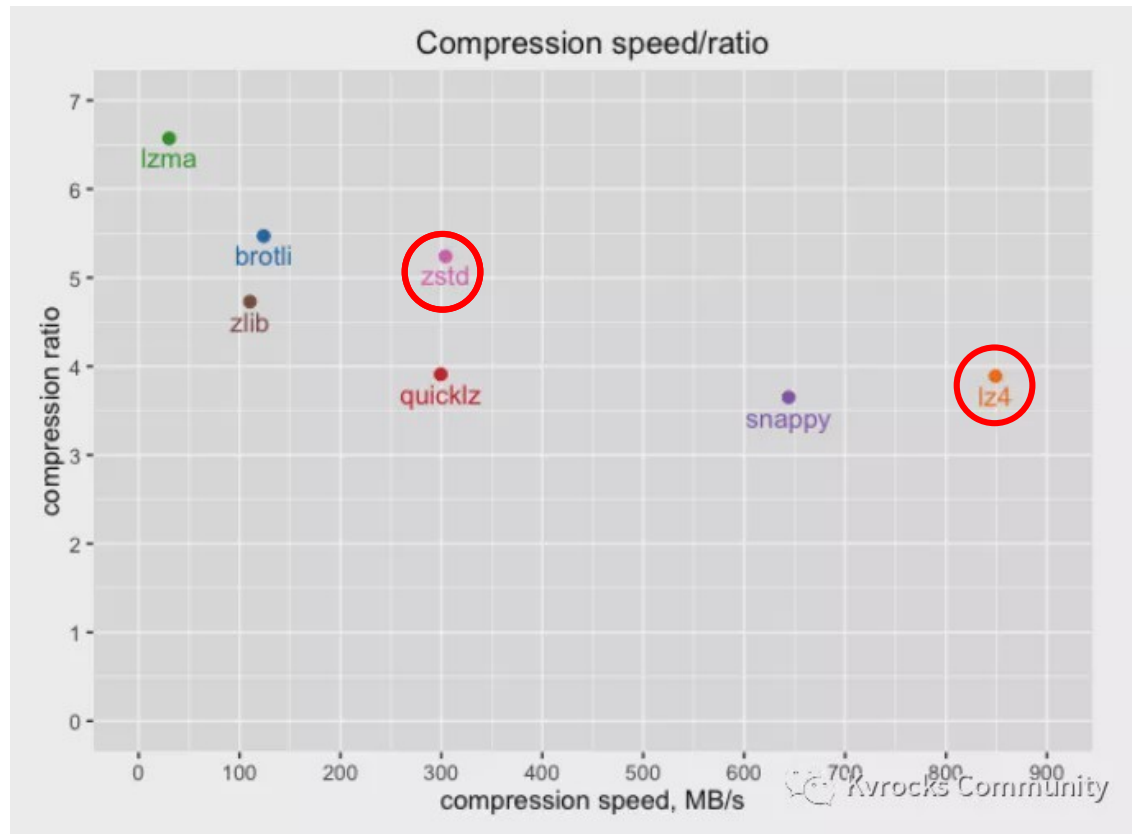
Log-structured merge-tree (LSM tree)



- SST = Sorted String Table, immutable
- WAL = Write Ahead Log
- minimizes disk IO
- writes large batches sequentially and compressed to disk
- designed for write intensive workloads

Compression

- Possible for
 - WAL (default no)
 - SST (default snappy)



Setup

- Docker compose:

```
kvrocks:
  image: 'apache/kvrocks:2.13.0'
  entrypoint: '/bin/kvrocks -c /etc/kvrocks.conf'
  ports:
    - '6379:6379'
  volumes:
    - 'kvrocks:/var/lib/kvrocks'
    - '._docker/kvrocks/kvrocks.conf:/etc/kvrocks.conf'
```

- Compiling the binary:

github.com/apache/kvrocks/blob/v2.13.0/Dockerfile

github.com/apache/kvrocks/blob/v2.13.0/utils/systemd/kvrocks.service

Configuration

```
workers 4 # number of worker threads
daemonize no
timeout 7200 # client idle timeout
rocksdb.block_cache_size 2048 # 2 GB, total memory usage 4-6 GB
rocksdb.compression lz4

dir /var/lib/kvrocks
log-dir /var/log/kvrocks
backup-dir /var/lib/kvrocks/backup

bind 0.0.0.0
port 6379
requirepass <password>
compact-cron "0 3 * * *" # removes deleted data, cleanup wal
compaction-checker-cron "* 0-7 * * *" # runs compact on files (if necessary)
bgsave-cron "0 2 * * *"

# defaults: https://github.com/apache/kvrocks/blob/unstable/kvrocks.conf
```


Kernel Configuration

Sysctl, kernel:

```
sudo sysctl -w vm.overcommit_memory=1  
sudo sysctl -w vm.swappiness=0  
sudo sysctl -w net.core.somaxconn=10000  
sudo sh -c "echo never > /sys/kernel/mm/transparent_hugepage/enabled"  
sudo sh -c "echo never > /sys/kernel/mm/transparent_hugepage/defrag"
```

Benchmark

workers: 2, ssd: wd sn740

redis-benchmark -t set,get -q

SET: 115473.45 requests per second, p50=0.215 msec

GET: 116414.43 requests per second, p50=0.215 msec

redis-benchmark -t get,set -q -P **8** # pipeline 8 requests

SET: 228832.95 requests per second, p50=1.687 msec # Redis 8.2: ~800k

GET: 840336.12 requests per second, p50=0.247 msec

redis-benchmark -t set,get -q -d **16384** # data size in bytes

SET: 22941.04 requests per second, p50=2.031 msec # Redis 8.2: ~80k

GET: 98039.22 requests per second, p50=0.223 msec

Summary

Apache KVRocks

- is Redis compatible

- works with data larger than ram

- uses a modern multi-core database architecture

- is suitable for write intensive workloads (sessions, queues, logs)

- is suitable for read intensive workloads (caches)

- works great in production

Resources

- [Apache KVRocks on GitHub](#)
- [Apache KVRocks Homepage](#)
- [Apache KVRocks supported commands](#)
- [RocksDB Homepage](#)
- [RocksDB Wikipedia](#)
- [RocksDB Block Cache](#)
- [LSM Tree Wikipedia](#)
- [LSM Trees: the Go-To Data Structure for Databases, Search Engines, and More](#)
- [UNIX socket vs TCP/IP ports](#)

Thank You for listening!

Questions?

Slides: github.com/thomas-0816/talks/

Mastodon: phpc.social/@thbley